SCHEME AND SOLUTION			
Course Code	EI243AI		
Sem	IV Semester	CIE-I	

MICROCONTROLLER AND PROGRAMMING

<table>
<tr><td></td><td>PART A</td></tr>
<tr><td>1 a)</td><td>ALU (1)</td></tr>
<tr><td>b)</td><td>HARWARD (1)</td></tr>
<tr><td>c)</td><td>APSR (1)</td></tr>
<tr><td>d)</td><td>Any random address
<table>
<tr><td colspan="2">Big-endian format: (1 mark)</td><td colspan="2">Little-endian(1 mark)</td></tr>
<tr><td>Address</td><td>Data</td><td>Address</td><td>data</td></tr>
<tr><td>0X100</td><td>0X12</td><td>0X100</td><td>0X78</td></tr>
<tr><td>0X101</td><td>0X34</td><td>0X101</td><td>0X56</td></tr>
<tr><td>0X102</td><td>0X56</td><td>0X102</td><td>0X34</td></tr>
<tr><td>0X103</td><td>0X78</td><td>0X103</td><td>0X12</td></tr>
</table></td></tr>
<tr><td>e)</td><td>The instruction STR r0, [r1, #12] in ARM assembly performs the following operation:
It stores the value in register r0 into the memory location pointed to by the address r1 + 12.
This means the value of r0 is saved to memory at the address formed by adding 12 to the value in r1.(1)</td></tr>
<tr><td>f)</td><td>Program Counter (PC): The Program Counter holds the memory address of the next instruction to be executed. It is automatically updated after each instruction to point to the next one.

Stack Pointer (SP): The Stack Pointer points to the top of the current stack in memory. It is used for managing function calls, local variables, and return addresses during program execution. The stack grows and shrinks as data is pushed to and popped from it. (0.5X2=1)</td></tr>
<tr><td>g)</td><td>R0= 0X02020202 (2 marks)</td></tr>
<tr><td>h)</td><td>Thumb2 1(marks)</td></tr>
<tr><td></td><td>PART B</td></tr>
<tr><td>1a.</td><td>Basic Block diagram involving below sub blocks (2Marks)

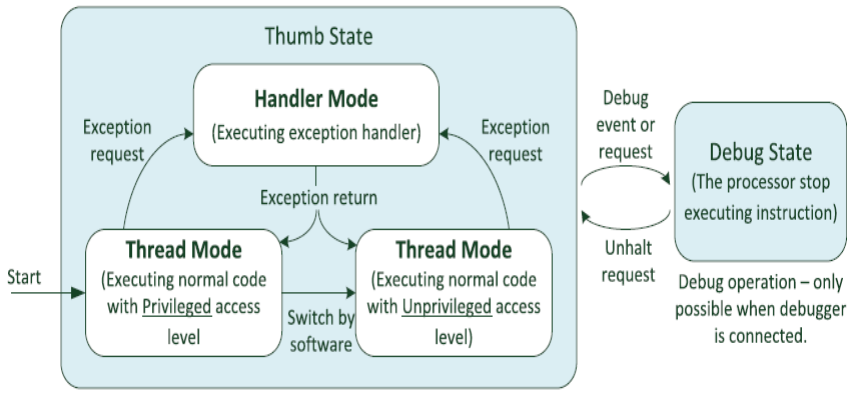
Explanation of The Control Unit (CU) (ALU, registers, memory)

Explanation of Instruction fetching
Instruction decoding control signal
Coordination (Theory 4 marks)</td></tr>
<tr><td>1b.</td><td>• CISC (Complex Instruction Set Computer):
o CISC architectures have a large number of instructions, which can perform complex operations in a single instruction (e.g., load, compute, and store in one command).
o CISC instructions are typically variable-length, allowing a single instruction to perform multiple operations.
o The focus is on minimizing the number of instructions to complete a task, making it more efficient for complex operations but often requiring more cycles to decode and execute each instruction.
o Example: x86 architecture.
• RISC (Reduced Instruction Set Computer):
o RISC architectures use a smaller set of instructions, with each instruction typically performing a simple, single operation (like adding two numbers or moving data).</td></tr>
</table>

	○ RISC instructions are fixed-length and simpler, designed to be executed in a single clock cycle. ○ The focus is on increasing instruction throughput by simplifying instructions and improving pipelining. ○ Example: **ARM** architecture. ANY 4 DIFFERENCE 4X1=4.
2a.	The **APSR (Application Program Status Register)** in the ARM Cortex M architecture contains flags that indicate the status of the processor after executing an instruction. It plays a crucial role in managing program state by holding flags that reflect the outcome of operations.(1) • **Flags in APSR**: ○ **N** (Negative): Set if the result of the operation was negative. ○ **Z** (Zero): Set if the result of the operation was zero. ○ **C** (Carry): Set if there was a carry out or borrow during an operation (for arithmetic). ○ **V** (Overflow): Set if there was an overflow during the operation.(4 marks) These flags are important for conditional execution of instructions and interrupt handling, as they allow the processor to adjust its behavior based on the result of the previous operations.(1) Explanation of each flags with examples
2b.	The **Thumb-2** instruction set is a combination of 16-bit and 32-bit instructions in the ARM Cortex M architecture. It offers the following benefits: • **Compact Code**: Thumb-2 provides 16-bit instructions that are more memory-efficient than the standard 32-bit ARM instructions. • **Flexibility**: It can dynamically switch between 16-bit and 32-bit instructions, making it more efficient for both compact code and complex operations. • **Performance**: By using 16-bit instructions for simpler tasks, Thumb-2 can increase instruction density (more instructions fit in the same memory space), and use 32-bit instructions when more computational power is needed.(3X1=3) Thus, Thumb-2 improves both performance and reduces memory usage, making it ideal for embedded systems where code size and memory resources are limited.(1 Mark)
3a	• **ARM Cortex-A**: ○ Designed for high-performance applications, such as smartphones, tablets, and other consumer electronics. ○ Supports complex instruction sets, virtualization, and multi-core processors. ○ Typically uses the ARMv7-A or ARMv8-A architecture with a focus on running operating systems like Linux or Android. • **ARM Cortex-R**: ○ Targeted at real-time applications where reliability and deterministic behavior are critical. ○ Used in applications like automotive safety, robotics, and industrial control systems. ○ Has features like low latency and support for error detection and correction. • **ARM Cortex-M**: ○ Designed for microcontroller applications, particularly in embedded systems with limited power and memory resources. ○ Focuses on low power consumption, efficient instruction execution, and simplicity. ○ Typically used in applications like IoT, wearables, and automotive control systems. 2x3=6 marks
3b	• **Sign bit (1 bit)**: Since the number is positive, the sign bit is 0.

	• **Convert the integer part (125) to binary**: 125 = 1111101. • **Convert the fractional part (.625) to binary**: ○ Result: .625 = .101. • **Combine the integer and fractional parts**: 125.625 = 1111101.101.(2 marks) • **Normalize the binary number**: 1.111101101 × 2^6. • **Exponent (8 bits)**: The exponent is 6 + bias (127 for single precision) = 133 = 10000101 in binary. • **Mantissa (23 bits)**: The mantissa is 111101101 followed by 0 bits to fill 23 bits, so the mantissa is 11110110100000000000000. Final representation in 32-bit single precision is: 0 10000101 11110110100000000000000 **0x42FB4000 (2 marks)**

4 a	. e **ARM Cortex-M** architecture is designed for embedded and real-time applications, offering an efficient instruction set and low-power features. These characteristics make it ideal for microcontrollers used in automotive systems, industrial automation, IoT devices, and medical electronics.
	
	Efficient Instruction Set
	
	1. **Thumb-2 Instruction Set**
	  ○ A mix of **16-bit and 32-bit instructions**, providing a balance between code density and performance.
	  ○ Reduces memory usage while maintaining high efficiency.
	2. **Hardware Division and DSP Instructions**
	  ○ Supports **single-cycle multiplication** and hardware division.
	  ○ Includes **Saturation and SIMD (Single Instruction Multiple Data)** for DSP-like operations.
	3. **Load/Store Architecture**
	  ○ Uses **register-based operations** to minimize memory accesses.
	  ○ Efficient stack handling and memory-mapped I/O for real-time applications.
	
	Low-Power Features
	
	1. **Multiple Low-Power Modes**
	  ○ **Sleep, Deep Sleep, and Standby modes** reduce power consumption.
	  ○ Dynamic frequency scaling allows power optimization.
	2. **Wake-up Interrupt Controller (WIC)**
	  ○ Enables peripherals to wake up the CPU only when necessary.
	3. **Energy-Efficient Pipeline**
	  ○ A **three-stage pipeline (Fetch, Decode, Execute)** reduces power usage while maintaining speed.
	
	Any 4 features 4X1=4 marks

4 b	 Block diagram 2 marks Explanation of Handler mode and Thumb mode 2 marks Explnation of Thumb state and Debug state 2 marks

5a	AREA BLOCK_SWAP, CODE, READONLY
	THUMB
	
	EXPORT main
	
	main
	LDR R0, =ARRAY1 ; Load base address of ARRAY1
	LDR R1, =ARRAY2 ; Load base address of ARRAY2 (2 marks)
	LDR R2, =ARRAY_SIZE ; Load block size
	MOV R3, #0 ; Initialize loop counter
	
	swap_loop
	CMP R3, R2 ; Compare counter with array size
	BGE done ; If counter >= size, exit
	
	; Swap elements
	LDRB R4, [R0, R3] ; Load byte from ARRAY1 into R4
	LDRB R5, [R1, R3] ; Load byte from ARRAY2 into R5 (2 marks)
	STRB R4, [R1, R3] ; Store byte from R4 into ARRAY2
	STRB R5, [R0, R3] ; Store byte from R5 into ARRAY1
	
	ADD R3, R3, #1 ; Increment counter
	B swap_loop ; Repeat loop
	
	done
	B done ; Infinite loop to halt execution
	
	; Data Section
	AREA DATA, DATA, READWRITE
	ARRAY1 DCB 10, 20, 30, 40, 50 ; Example array 1
	ARRAY2 DCB 60, 70, 80, 90, 100 ; Example array 2 (2 marks)
	ARRAY_SIZE EQU 5 ; Size of arrays (bytes)
	END
	
	Depend on assembly directive DCB OR DCD OR DCW need to consider the appropriate logic.

5 b	LDR (Load Register) The LDR instruction loads data from a memory location into a register. It can be used with both immediate values (like constants) or registers (with an address).

	LDR <register>, [<address>]
	Where:
	
	<register>: The register that will receive the data.
	
	<address>: The memory location, which can be specified by a register (with or without an offset).
	
	LDR R0, =0x20001000 ; Load the immediate address 0x20001000 into R0
	
	LDR R1, [R0] ; Load the value at the memory address 0x20001000 into R1
	
	The **STR** instruction stores the value from a register into memory.
	STR <register>, [<address>]
	LDR R0, =0x20001000 ; Load the address 0x20001000 into R0
	MOV R1, #42 ; Load the value 42 into R1
	STR R1, [R0] ; Store the value in R1 (42) into memory at address 0x20001000